

Programación para Física y Astronomía

Departamento de Física.

Coordinadora: C Loyola

Profesores C Femenías / D Basantes

Primer Semestre 2025

Universidad Andrés Bello

Departamento de Física y Astronomía



Introducción y Conexión

Ejercicios Prácticos

Soluciones de Referencia

Consolidación y Retroalimentación

Conclusiones

Apéndice: Recursos Adicionales

Introducción y Conexión

Conceptos fundamentales que aprendimos

- Ajuste con **np.polyfit**: ajuste de polinomios por mínimos cuadrados
- Métricas de calidad: Residuos, R^2 , χ^2_{red}
- Ponderación: Uso de pesos $w = 1/\sigma$
- Ejemplo físico: Ley de Hubble

Objetivo de hoy (P2)

Aplicar ajustes lineales con **np.polyfit** en problemas prácticos de física.

Aplicación de ajustes lineales:

1. **Ejercicio 1:** Calibración de sensor de temperatura

- Ajuste lineal simple: $V = aT + b$
- Cálculo de R^2

2. **Ejercicio 2:** Ley de Hubble con análisis completo

- Ajuste con ponderación
- Cálculo de χ^2_{red}
- Gráficos de residuos

Sesión práctica enfocada en dominar `np.polyfit`!

Ejercicios Prácticos



Enunciado

- Un sensor de temperatura mide voltaje V según $V = aT + b$
- Simule 12 medidas en $T = 0, 10, 20, \dots, 110^\circ\text{C}$
- Use $a_{\text{real}} = 0.018 \text{ V}/^\circ\text{C}$, $b_{\text{real}} = 0.15 \text{ V}$
- Agregue ruido gaussiano $\sigma = 0.02 \text{ V}$ (constante)
- **Tareas:**
 1. Ajuste con `np.polyfit(T, V, 1)`
 2. Calcule y reporte R^2
 3. Grafique datos y recta ajustada
 4. Compare a_{est} y b_{est} con valores reales

Conceptos: Ajuste lineal básico, R^2



Enunciado

- Relación de Hubble: $v = H_0 d + b$
- Simule 25 galaxias con $d \in [10, 200]$ Mpc (espaciado uniforme)
- Use $H_0 = 70.0$ km/s/Mpc, $b = 0$ km/s
- Agregue ruido $\sigma_v = 180$ km/s (velocidades peculiares)
- Tareas:
 1. Ajuste con ponderación: `np.polyfit(d, v, 1, w=1/sigma_v)`
 2. Calcule χ^2_{red}
 3. Grafique: (a) datos + ajuste con barras de error, (b) residuos vs distancia
 4. Interprete: ¿es $\chi^2_{\text{red}} \approx 1$?

Conceptos: Ponderación, χ^2_{red} , residuos, aplicación cosmológica

Soluciones de Referencia



```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Generar datos sintéticos
5 np.random.seed(42)
6 T = np.arange(0, 120, 10)
7 a_true, b_true = 0.018, 0.15
8 sigma = 0.02
9 V = a_true*T + b_true + \
10     np.random.normal(0, sigma,
11                     len(T))
12
13 # Ajustar con polyfit
14 a_est, b_est = np.polyfit(T, V, 1)
15 V_fit = a_est*T + b_est
```

```
16 # Calcular R^2
17 SS_res = np.sum((V - V_fit)**2)
18 SS_tot = np.sum((V - np.mean(V))**2)
19 R2 = 1 - SS_res/SS_tot
20
21 # Graficar
22 plt.scatter(T, V, label='Datos')
23 plt.plot(T, V_fit, 'r',
24         label=f'Ajuste: '
25             f'a={a_est:.4f}, '
26             f'b={b_est:.3f}')
27 plt.xlabel('Temperatura (°C)')
28 plt.ylabel('Voltaje (V)')
29 plt.legend(); plt.grid()
30 plt.show()
31
32 print(f"a = {a_est:.4f} V/°C")
33 print(f"b = {b_est:.3f} V")
34 print(f"R² = {R2:.4f}")
```



Ley de Hubble (Parte 1: Ajuste)

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Generar datos
5 np.random.seed(10)
6 H0_true, sigma_v = 70.0, 180.0
7 d = np.linspace(10, 200, 25)
8 v_obs = H0_true*d + \
9     np.random.normal(0, sigma_v,
10                     len(d))
11
12 # Ajuste CON ponderación
13 w = 1.0 / sigma_v
14 H0_est, b_est = np.polyfit(
15     d, v_obs, 1, w=w)
16 v_fit = H0_est*d + b_est
```

```
17 # Calcular chi^2_red
18 res = v_obs - v_fit
19 chi2 = np.sum((res/sigma_v)**2)
20 nu = len(d) - 2
21 chi2_red = chi2/nu
22
23 print(f"H0 = {H0_est:.1f} "
24       f"km/s/Mpc")
25 print(f"b = {b_est:.1f} km/s")
26 print(f"χ²_red = {chi2_red:.2f}")
27
28 # Interpretación
29 if 0.5 < chi2_red < 2.0:
30     print("✅ Ajuste consistente")
31 else:
32     print("❌ Revisar modelo")
```

Soluciones de Referencia $\ni$ Solución 2:

Ley de Hubble (Parte 2: Visualización)



```
1 # Gráfico principal
2 plt.figure(figsize=(10, 4))
3
4 plt.subplot(1, 2, 1)
5 plt.errorbar(d, v_obs,
6             yerr=sigma_v,
7             fmt='o', capsize=3,
8             label='Galaxias')
9 plt.plot(d, v_fit, 'r-',
10         label=f'H0={H0_est:.1f}')
11 plt.xlabel('Distancia (Mpc)')
12 plt.ylabel('Velocidad (km/s)')
13 plt.legend(); plt.grid()
```

```
13 # Gráfico de residuos
14 plt.subplot(1, 2, 2)
15 plt.scatter(d, res, alpha=0.6)
16 plt.axhline(0, color='r',
17            linestyle='--')
18 plt.axhline(sigma_v, color='gray',
19            linestyle=':',
20            label='±σ')
21 plt.axhline(-sigma_v,
22            color='gray',
23            linestyle=':')
24 plt.xlabel('Distancia (Mpc)')
25 plt.ylabel('Residuos (km/s)')
26 plt.legend(); plt.grid()
27
28 plt.tight_layout()
29 plt.show()
```

Consolidación y Retroalimentación

Reflexión sobre los ejercicios

- ¿Lograron implementar `np.polyfit` correctamente?
- ¿Cómo interpretaron el R^2 y el χ^2_{red} ?
- ¿Qué les pareció el uso de ponderación (parámetro w)?
- ¿Cómo interpretaron las barras de error y los residuos?

Objetivo

Desarrollar criterio físico-estadístico al evaluar ajustes lineales con incertidumbres.

Errores frecuentes

- Olvidar el grado del polinomio: `np.polyfit(x, y, 1)` para lineal
- No revisar el χ^2_{red} : R^2 alto no garantiza consistencia con incertidumbres
- Ignorar el parámetro **w**: Esencial cuando hay ponderación
- No graficar residuos: Patrones sistemáticos indican modelo inadecuado

Consejo

Workflow: Visualizar datos $\rightarrow$ Ajustar con `np.polyfit` $\rightarrow$ Calcular R^2 y χ^2_{red}
 $\rightarrow$ Graficar residuos $\rightarrow$ Interpretar

Conclusiones

Lo que aprendimos hoy

- **Uso de `np.polyfit`:** Ajustes lineales y polinomiales
- **Métricas de calidad:** Cálculo de R^2 y χ^2_{red}
- **Ponderación:** Uso del parámetro w para incertidumbres
- **Análisis de residuos:** Diagnóstico visual de la calidad del ajuste
- **Aplicaciones físicas:** Calibración de sensores y Ley de Hubble

Habilidad adquirida

Realizar ajustes lineales rigurosos con `np.polyfit`, evaluando calidad mediante métricas estadísticas.

Funciones clave de NumPy

- `np.polyfit(x, y, deg, w=None)`: Ajuste polinomial (lineal cuando `deg=1`)
- `np.mean()`, `np.sum()`: Cálculo de estadísticas
- `np.linspace()`, `np.arange()`: Generación de arrays
- `matplotlib.pyplot`: Visualización de datos y residuos

Métricas implementadas

- R^2 : Bondad de ajuste general ($R^2 = 1 - SS_{\text{res}}/SS_{\text{tot}}$)
- χ^2_{red} : Consistencia con incertidumbres ($\chi^2_{\text{red}} = \chi^2/\nu$)
- Residuos: $r_i = y_i - \hat{y}_i$ para diagnóstico visual

- **Próxima semana (Semana 12):** Ajustes no lineales con `curve_fit`
- **Recomendación de estudio:**
 - Repasar `np.polyfit` y las métricas vistas hoy
 - Explorar documentación de `scipy.optimize.curve_fit`
 - Revisar funciones no lineales comunes en física
- **Práctica sugerida:**
 - Repetir ejercicios con diferentes datasets
 - Experimentar con diferentes grados de polinomios

¡Nos vemos en la próxima sesión!

Apéndice: Recursos Adicionales

```
1 import numpy as np
2
3 # Datos de ejemplo
4 x = np.linspace(0, 10, 30)
5 y_true = 2.0*x + 1.0
6 y = y_true + np.random.normal(0, 0.5, len(x))
7
8 # Ajuste
9 m, b = np.polyfit(x, y, 1)
10
11 # Calcular  $R^2$  y métricas
12 y_fit = m*x + b
13 SS_res = np.sum((y - y_fit)**2)
14 SS_tot = np.sum((y - np.mean(y))**2)
15 R2 = 1 - SS_res/SS_tot
16
17 print(f"m = {m:.2f}, b = {b:.2f}")
18 print(f"R² = {R2:.4f}")
```

Nota: Para ajustes más complejos y errores formales de parámetros, ver `curve_fit` en Semana 12.